

Implementazione e analisi sperimentale dell'algoritmo di Brandes per la betweenness centrality

Sebastiano Becchetti

1 luglio 2026

Indice

1	Introduzione al problema e soluzioni precedenti	2
1.1	La betweenness centrality	2
1.2	L'approccio classico e il suo costo	2
1.3	Implementazione e limiti dell'approccio standard	3
1.4	Il contributo di Brandes	5
1.5	Obiettivo	5
2	Implementazione proposta	6
2.1	La relazione ricorsiva sulle dipendenze	6
2.2	Dimostrazione della relazione ricorsiva	7
2.3	Pseudocodice	11
2.4	Esempio passo-passo: il diamante a 5 vertici	11
2.5	Generazione dei grafi di test	14
2.6	Il calcolo dei cammini minimi	14
2.7	L'accumulo all'indietro	15
2.8	Ottimizzazioni implementative	15
2.8.1	Rappresentazione dell'adiacenza: liste contro CSR	16
3	Estensione ai grafi pesati: l'algoritmo di Dijkstra	18
3.1	Inadeguatezza della BFS	18
3.2	Implementazione con pesi non unitari	18
3.3	La cancellazione pigra	19
3.4	L'accumulo all'indietro nel caso pesato	19
3.5	Complessità	20
3.6	Verifica ed esempio visivo	21
4	Risultati sperimentali	23
4.1	Impostazione degli esperimenti	23
4.2	La pipeline sperimentale	23
4.3	Verifica di correttezza	24
4.4	Interpretazione visiva della betweenness	25
4.5	Analisi della complessità	25
4.5.1	Il sovraccarico lineare per sorgente	25
4.6	Riproduzione dell'esperimento di Brandes	26
4.7	Confronto sperimentale con l'approccio standard	27
5	Conclusioni	29
5.1	Sintesi del lavoro	29

Capitolo 1

Introduzione al problema e soluzioni precedenti

1.1 La betweenness centrality

Nell'analisi delle reti reali — sociali, biologiche, infrastrutturali — ricorre l'esigenza di quantificare la rilevanza di un singolo nodo per la costruzione del grafo, e quindi quanto sia importante per la costruzione dei cammini minimi tra i vari nodi. Tra le misure proposte a tale scopo, una delle più diffuse è la *betweenness centrality*, formalizzata da Freeman [2] a partire da idee precedentemente introdotte da Anthonisse [3].

Il principio su cui si fonda la misura è il seguente: la centralità di un nodo è tanto maggiore quanto più frequentemente esso è presente sui cammini minimi che collegano le altre coppie di nodi. Un nodo con *betweenness* elevata costituisce un punto di passaggio pressoché obbligato, la cui rimozione o congestione compromette la comunicazione tra ampie porzioni della rete.

Formalmente, dato un grafo $G = (V, E)$ con $n = |V|$ vertici e $m = |E|$ archi, si definisce la *betweenness centrality* di un vertice v come

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}, \quad (1.1)$$

dove σ_{st} è il numero totale di cammini minimi tra s e t , mentre $\sigma_{st}(v)$ è il numero di quei cammini che passano per v . Il rapporto $\sigma_{st}(v)/\sigma_{st}$ esprime la frazione di cammini minimi tra s e t che dipendono da v : in presenza di più cammini minimi alternativi, il contributo di v viene ripartito di conseguenza.

1.2 L'approccio classico e il suo costo

Il calcolo diretto della formula (1.1) richiede, per ogni coppia di vertici (s, t) , di conoscere il numero e la composizione dei cammini minimi. Gli approcci classici procedono in due fasi:

1. calcolo di tutte le distanze e di tutti i cammini minimi tra tutte le coppie (*all-pairs shortest paths*);
2. somma esplicita, per ogni coppia (s, t) e per ogni vertice intermedio v , del contributo $\sigma_{st}(v)/\sigma_{st}$.

La seconda fase costituisce il principale fattore limitante. La somma esplicita richiede, per ciascuno degli n vertici v , di scorrere tutte le $\Theta(n^2)$ coppie ordinate (s, t) : il corpo interno viene quindi eseguito $\Theta(n^3)$ volte, da cui il costo temporale $\Theta(n^3)$. A ciò si aggiunge il costo

in memoria: per valutare ogni rapporto $\sigma_{st}(v)/\sigma_{st}$ occorre aver memorizzato i conteggi σ_{st} per tutte le coppie di vertici, ovvero uno spazio $O(n^2)$. Per reti dell'ordine di decine di migliaia di nodi è proprio l'occupazione di memoria quadratica a divenire proibitiva, ancor prima del costo temporale. La sezione seguente ne presenta l'implementazione, il relativo pseudocodice e i tempi misurati.

1.3 Implementazione e limiti dell'approccio standard

Per misurarne concretamente i limiti, l'approccio classico è stato implementato in C++ (file `src/standard.cpp`), seguendo fedelmente le due fasi:

- **Fase 1 (all-pairs):** una BFS da ciascuna sorgente calcola e *memorizza* le matrici complete delle distanze $d[s][t]$ e dei conteggi di cammini minimi $\sigma[s][t]$ per tutte le coppie;
- **Fase 2 (somma esplicita):** per ogni vertice intermedio v e per ogni coppia (s, t) , il *criterio di Bellman* stabilisce se v giace su un cammino minimo da s a t , nel qual caso si accumula la relativa dipendenza di coppia.

Algorithm 1 Approccio standard: all-pairs seguito da somma esplicita.

```

1: Fase 1: cammini minimi tra tutte le coppie
2: for  $s \in V$  do ▷ una BFS per sorgente
3:    $d[s][t] \leftarrow -1, \sigma[s][t] \leftarrow 0$  per ogni  $t$ 
4:    $d[s][s] \leftarrow 0, \sigma[s][s] \leftarrow 1$ 
5:   BFS da  $s$  che aggiorna  $d[s][\cdot]$  e  $\sigma[s][\cdot]$ 
6: end for
7: Fase 2: somma esplicita delle dipendenze di coppia
8:  $C_B[v] \leftarrow 0$  per ogni  $v \in V$ 
9: for  $v \in V$  do
10:  for  $s \in V, s \neq v$  do
11:    for  $t \in V, t \neq s, t \neq v$  do
12:      if  $d[s][v] + d[v][t] = d[s][t]$  then ▷ criterio di Bellman
13:         $C_B[v] += \frac{\sigma[s][v] \cdot \sigma[v][t]}{\sigma[s][t]}$ 
14:      end if
15:    end for
16:  end for
17: end for
18: per grafi non orientati:  $C_B[v] \leftarrow C_B[v]/2$  per ogni  $v$ 

```

Si ritrovano i due costi: la Fase 2 è $\Theta(n^3)$ (tre cicli su V), mentre le matrici d e σ della Fase 1 impongono uno spazio $\Theta(n^2)$. Il vero collo di bottiglia è quest'ultimo: già per reti di poche decine di migliaia di nodi le due matrici non entrano in memoria, rendendo l'approccio inapplicabile indipendentemente dal tempo.

La Figura 1.1 riporta i tempi misurati dall'implementazione, con lo stesso schema sperimentale usato nel seguito (una curva per densità, sweep su n). Il range di n è qui limitato a 1000: il costo $\Theta(n^3)$ rende già proibitivo spingersi oltre.

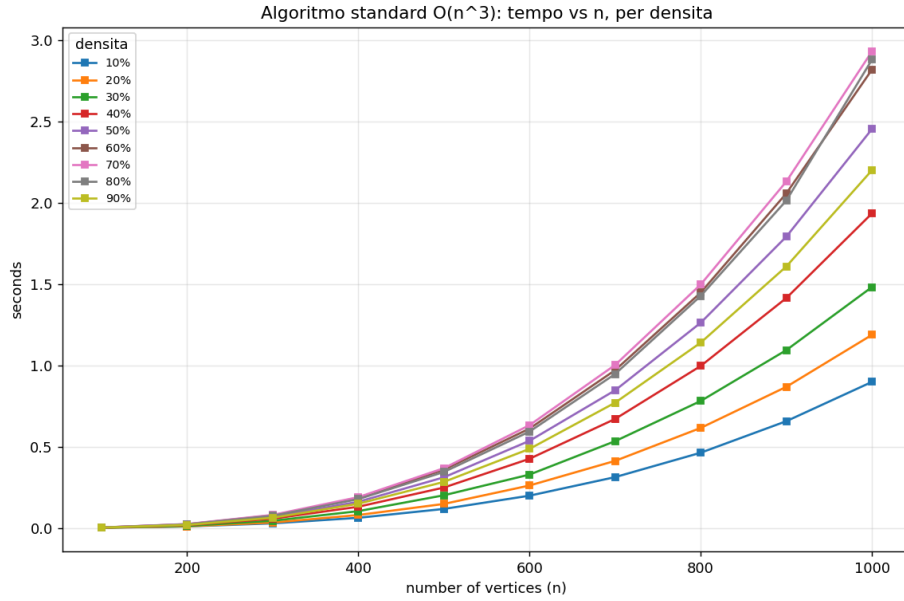


Figura 1.1: Tempo di esecuzione dell'approccio standard $O(n^3)$ in funzione del numero di vertici, una curva per ciascuna densità.

Il confronto diretto con l'algoritmo di Brandes (Figura 1.2) chiarisce dove stia il vantaggio. Sui grafi qui generati, a densità 10%–90%, si ha $m = \Theta(n^2)$: in questo regime denso anche Brandes costa $O(nm) = O(n^3)$, e i tempi sono dunque dello stesso ordine (lo standard resta più lento per la costante della somma esplicita). Il guadagno decisivo di Brandes emerge invece sui grafi *sparsi* — tipici delle reti reali — e, soprattutto, nello spazio: $O(n + m)$ lineare contro $O(n^2)$.

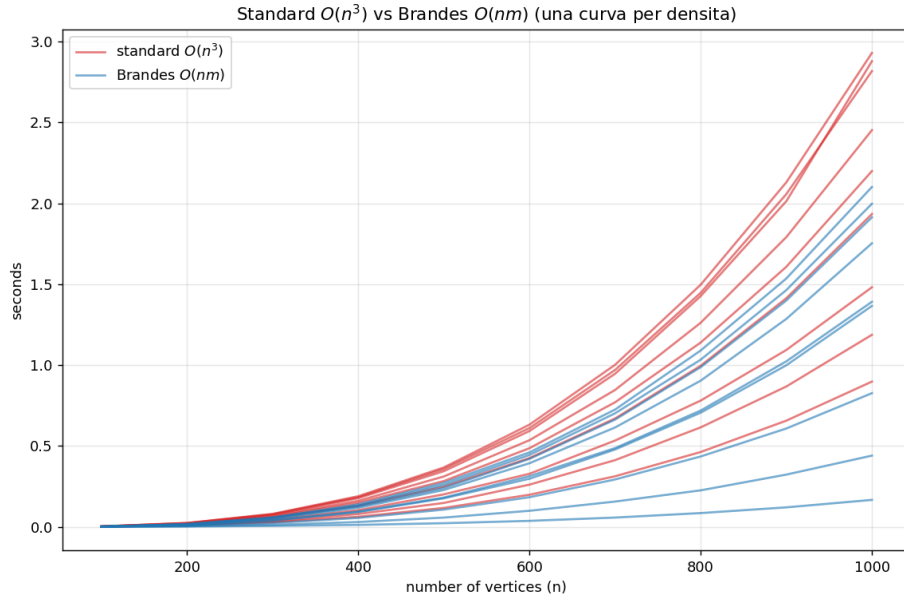


Figura 1.2: Confronto dei tempi: approccio standard $O(n^3)$ (rosso) contro algoritmo di Brandes $O(nm)$ (blu), una curva per densità sul range di n comune.

1.4 Il contributo di Brandes

Nel 2001 Brandes [1] propone un algoritmo che riduce sensibilmente il costo computazionale senza introdurre alcuna approssimazione. L'idea centrale consiste nel non sommare le dipendenze coppia per coppia, bensì nell'*accumularle* in forma aggregata considerando una sola sorgente per volta, attraverso una relazione ricorsiva sulle dipendenze discussa nel Capitolo 2.

I costi ottenuti sono:

- **grafi non pesati:** $O(nm)$ tempo, tramite una BFS (*Breadth-First Search*) da ciascuna sorgente;
- **grafi pesati:** $O(nm + n^2 \log n)$ tempo, usando l'algoritmo di Dijkstra con *Fibonacci heap* [4];
- **spazio:** $O(n + m)$ in entrambi i casi, ovvero lineare.

La riduzione dello spazio da $O(n^2)$ a $O(n + m)$ è l'elemento che rende l'algoritmo applicabile a reti di grande dimensione.

1.5 Obiettivo

La presente tesina documenta un'implementazione in C++ dell'algoritmo di Brandes per grafi non orientati, ne verifica la correttezza mediante confronto con la libreria NetworkX [5] e ne analizza sperimentalmente la complessità, riproducendo lo schema di esperimento presentato nell'articolo originale. La trattazione muove dal caso *non pesato* (Capitolo 2), in cui i cammini minimi sono determinati tramite la BFS, e si estende successivamente al caso *pesato* (Capitolo 3), in cui la costruzione dei cammini minimi è sostituita dall'algoritmo di Dijkstra; di quest'ultima variante si illustrano il funzionamento, le modifiche apportate al codice e l'andamento sperimentale dei tempi di esecuzione al variare della densità. Vengono inoltre discusse alcune ottimizzazioni a costante (riutilizzo dei buffer di memoria) che, pur non modificando la complessità asintotica $O(nm)$, riducono sensibilmente i tempi di esecuzione.

Capitolo 2

Implementazione proposta

Il presente capitolo descrive in dettaglio l'algoritmo di Brandes per grafi non pesati, ponendo ciascun passo dello pseudocodice in corrispondenza con la relativa porzione del codice C++ contenuto nel file `src/brandes.cpp`.

2.1 La relazione ricorsiva sulle dipendenze

Il fulcro dell'algoritmo è la nozione di *dipendenza* di una sorgente s da un vertice v :

$$\delta_s(v) = \sum_{t \in V} \frac{\sigma_{st}(v)}{\sigma_{st}}. \quad (2.1)$$

Sommando $\delta_s(v)$ su tutte le sorgenti si riottiene $C_B(v) = \sum_s \delta_s(v)$. Brandes dimostra che la dipendenza soddisfa la relazione ricorsiva

$$\delta_s(v) = \sum_{w: v \in P_s(w)} \frac{\sigma_{sv}}{\sigma_{sw}} (1 + \delta_s(w)), \quad (2.2)$$

dove $P_s(w)$ è l'insieme dei *predecessori* di w lungo i cammini minimi da s , cioè i vertici v adiacenti a w tali che $d(s, v) + 1 = d(s, w)$.

È istruttivo soffermarsi sul fattore $(1 + \delta_s(w))$, che è il cuore della relazione. Conviene però partire dal coefficiente σ_{sv}/σ_{sw} . A numeratore σ_{sv} è il numero di cammini minimi da s a v , a denominatore σ_{sw} è il numero di cammini minimi da s a w (non quelli che *passano* per w , ma quelli che vi *arrivano*). Poiché v è un predecessore di w , ogni cammino minimo da s a w che transita per v si ottiene prolungando di un arco un cammino minimo da s a v : i cammini minimi verso w passanti per v sono dunque esattamente σ_{sv} . Il rapporto σ_{sv}/σ_{sw} è perciò la *frazione* dei cammini minimi diretti a w che lo raggiungono passando per v , ovvero la probabilità che, scelto a caso uno dei cammini minimi $s \rightarrow w$, esso transiti per v . Non è quindi una misura di “quanto v dipende da w ”, bensì la quota di instradamento verso w che spetta a v : è il peso con cui v ha diritto a ereditare la dipendenza di w . Questa frazione viene moltiplicata per due contributi distinti. Il termine 1 rappresenta la dipendenza dovuta alla coppia (s, w) *stessa*: v giace su un cammino minimo da s a w , dunque va accreditato come vertice intermedio rispetto alla destinazione w . Il termine $\delta_s(w)$, invece, propaga a v tutta la dipendenza già accumulata da w : poiché w sta sui cammini minimi verso ogni destinazione t *più lontana di* w , e v sta sui cammini minimi verso w , allora v giace anche su tutti i cammini minimi $s \rightarrow \dots \rightarrow v \rightarrow w \rightarrow \dots \rightarrow t$. In altre parole, v non viene accreditato soltanto per i cammini che terminano in w , ma per ogni cammino minimo che parte da s , passa per v e per w , e prosegue oltre verso un qualunque vertice t a valle di w . È esattamente questa propagazione ricorsiva — ogni successore w cede a v la propria dipendenza più l'unità

corrispondente a sé stesso come destinazione — che consente di evitare la somma esplicita coppia per coppia.

La relazione (2.2) permette quindi di calcolare tutte le dipendenze di una sorgente con una sola passata all'indietro sui vertici, in ordine di distanza non crescente.

2.2 Dimostrazione della relazione ricorsiva

La relazione (2.2) è il risultato centrale su cui poggia l'algoritmo: ne giustifichiamo qui la validità, muovendo dalle definizioni di base e appoggiandoci a due lemmi di natura combinatoria sui conteggi dei cammini minimi. La dimostrazione segue quella originale di Brandes [1].

Definizioni

Per una coppia ordinata di vertici distinti (s, t) si denotano con σ_{st} il numero di cammini minimi da s a t e con $\sigma_{st}(v)$ il numero di quelli che transitano per un vertice intermedio $v \neq s, t$; $d(s, t)$ è la distanza fra s e t . La *betweenness centrality* di v è

$$C_B(v) = \sum_{s \neq v} \sum_{t \neq s, t \neq v} \frac{\sigma_{st}(v)}{\sigma_{st}}, \quad (2.3)$$

e la quantità $\delta_{st}(v) = \sigma_{st}(v)/\sigma_{st}$, detta *dipendenza di coppia*, esprime la frazione dei cammini minimi $s \rightarrow t$ che passano per v . Raggruppando i contributi a parità di sorgente si ottiene la *dipendenza* $\delta_s(v) = \sum_t \delta_{st}(v)$ già introdotta, per la quale vale $C_B(v) = \sum_{s \neq v} \delta_s(v)$. Il calcolo diretto della (2.3) richiederebbe di conoscere i $\Theta(n^3)$ valori $\sigma_{st}(v)$; l'intera efficienza dell'algoritmo discende dal non calcolarli mai esplicitamente, ma dal ricavare le dipendenze tramite la (2.2).

Si chiama infine *insieme dei predecessori* di w rispetto a s

$$P_s(w) = \{ u \in V : \{u, w\} \in E, d(s, w) = d(s, u) + 1 \},$$

cioè l'insieme dei vertici da cui parte l'ultimo arco di un cammino minimo da s a w ; i predecessori inducono il *grafo aciclico dei cammini minimi* radicato in s .

Due lemmi sui conteggi

Lemma 1 (Conteggio dei cammini minimi). *Per ogni $w \neq s$ vale $\sigma_{sw} = \sum_{u \in P_s(w)} \sigma_{su}$.*

Dimostrazione. Ogni cammino minimo da s a w ha lunghezza $d(s, w)$ e termina con un arco $\{u, w\}$ tale che $d(s, u) = d(s, w) - 1$, ossia $u \in P_s(w)$. Sopprimendone l'ultimo arco si ottiene un cammino minimo da s a u , e tale operazione stabilisce una corrispondenza biunivoca fra i cammini minimi $s \rightarrow w$ aventi u come penultimo vertice e i cammini minimi $s \rightarrow u$. Al variare di u in $P_s(w)$ questi insiemi formano una partizione dei cammini minimi $s \rightarrow w$, da cui la somma.

Il Lemma 1 è esattamente la regola con cui la BFS aggiorna i contatori σ : alla scoperta di un cammino minimo verso w passante per v si pone $\sigma[w] += \sigma[v]$.

Lemma 2 (Fattorizzazione di un vertice intermedio). *Se v giace su almeno un cammino minimo da s a t , allora $\sigma_{st}(v) = \sigma_{sv} \cdot \sigma_{vt}$.*

Dimostrazione. Poiché v è sul cammino minimo, $d(s, t) = d(s, v) + d(v, t)$: ogni cammino minimo $s \rightarrow t$ passante per v si scompone in modo unico nella concatenazione di un cammino minimo $s \rightarrow v$ e di un cammino minimo $v \rightarrow t$. Viceversa, ogni coppia formata da un cammino minimo $s \rightarrow v$ e da un cammino minimo $v \rightarrow t$, accostata, dà un cammino minimo $s \rightarrow t$ passante per v . Il conteggio di un prodotto cartesiano è il prodotto dei conteggi.

Dal Lemma 2 segue immediatamente la forma esplicita della dipendenza di coppia, $\delta_{st}(v) = \frac{\sigma_{sv} \sigma_{vt}}{\sigma_{st}}$.

Il teorema ricorsivo

Teorema 1 (Brandes, 2001). *Per ogni $v \neq s$ vale*

$$\delta_s(v) = \sum_{w: v \in P_s(w)} \frac{\sigma_{sv}}{\sigma_{sw}} (1 + \delta_s(w)),$$

dove la somma è estesa ai successori di v nel grafo dei cammini minimi, cioè ai vertici w di cui v è predecessore.

Dimostrazione. Si fissi $v \neq s$ e un bersaglio $t \neq s, v$. In ogni cammino minimo da s a t che passa per v , il vertice immediatamente successivo a v è un vicino w che ne è successore nel grafo dei cammini minimi, cioè $v \in P_s(w)$. Classificando tali cammini in base a questo primo vertice w si ottiene una partizione, da cui

$$\sigma_{st}(v) = \sum_{w: v \in P_s(w)} \sigma_{st}(v, w),$$

avendo indicato con $\sigma_{st}(v, w)$ il numero di cammini minimi $s \rightarrow t$ che passano per v e ne escono verso w .

Questa quantità si fattorizza. Per $t \neq w$, un simile cammino è la concatenazione di un cammino minimo $s \rightarrow w$ avente v come penultimo vertice con un cammino minimo $w \rightarrow t$; i primi sono σ_{sv} (Lemma 1, essendo $v \in P_s(w)$), sicché, usando $\sigma_{st}(w) = \sigma_{sw} \sigma_{wt}$ (Lemma 2),

$$\sigma_{st}(v, w) = \sigma_{sv} \sigma_{wt} = \frac{\sigma_{sv}}{\sigma_{sw}} \sigma_{st}(w).$$

L'uguaglianza resta valida anche quando w non giace su alcun cammino minimo $s \rightarrow t$: in tal caso $\sigma_{st}(w) = 0$ ed entrambi i membri si annullano. Per $t = w$, infine, i cammini minimi $s \rightarrow w$ con penultimo vertice v sono esattamente σ_{sv} , e il loro contributo a $\delta_s(v)$ è σ_{sv}/σ_{sw} .

Sostituendo nella definizione di $\delta_s(v)$ e scambiando l'ordine delle somme,

$$\delta_s(v) = \sum_{t \neq s, v} \frac{\sigma_{st}(v)}{\sigma_{st}} = \sum_{w: v \in P_s(w)} \left(\underbrace{\frac{\sigma_{sv}}{\sigma_{sw}}}_{t=w} + \sum_{t \neq s, w} \frac{1}{\sigma_{st}} \cdot \frac{\sigma_{sv}}{\sigma_{sw}} \sigma_{st}(w) \right).$$

Nel secondo addendo si raccoglie il fattore costante e si riconosce la definizione di dipendenza:

$$\frac{\sigma_{sv}}{\sigma_{sw}} \sum_{t \neq s, w} \frac{\sigma_{st}(w)}{\sigma_{st}} = \frac{\sigma_{sv}}{\sigma_{sw}} \delta_s(w).$$

Sommando i due contributi relativi a ciascun successore w si ottiene $\frac{\sigma_{sv}}{\sigma_{sw}} (1 + \delta_s(w))$ e, sommando su tutti i successori, la (2.2).

Conseguenza per l'algoritmo

Nel Teorema 1 la dipendenza di v è espressa in funzione delle sole dipendenze $\delta_s(w)$ dei suoi successori, per i quali $d(s, w) > d(s, v)$. La ricorsione è dunque *ben fondata*: elaborando i vertici in ordine di distanza non crescente, quando si tratta v tutti i valori $\delta_s(w)$ necessari sono già stati calcolati. È precisamente ciò che garantisce la pila S , che restituisce i vertici nell'ordine inverso a quello di inserimento durante la BFS. L'algoritmo realizza la somma “per successori” come una distribuzione “ai predecessori”: estratto w da S , ne riversa il contributo $\frac{\sigma_{sv}}{\sigma_{sw}}(1 + \delta_s(w))$ su ciascun $v \in P_s(w)$. Accumulando infine $C_B[w] += \delta_s(w)$ per ogni sorgente s , il vettore C_B riproduce esattamente la doppia somma (2.3); nel caso non orientato ogni coppia non ordinata è conteggiata due volte, da cui la divisione finale per 2.

Vale la pena osservare che il calcolo delle dipendenze è, a tutti gli effetti, un problema di *programmazione dinamica*. Ne possiede infatti i due tratti caratteristici: la *sottostruttura ottima*, espressa dalla relazione (2.2), che riconduce $\delta_s(v)$ ai valori $\delta_s(w)$ dei successori; e la *sovrapposizione dei sottoproblemi*, poiché la dipendenza $\delta_s(w)$ di un vertice è condivisa da tutti i suoi predecessori e dovrebbe essere ricalcolata molte volte se affrontata in modo ingenuo. Risolvere ogni sottoproblema una sola volta e riutilizzarne il risultato — memorizzandolo nel vettore δ ed elaborando i vertici nell'ordine topologico indotto dalla distanza decrescente (l'ordine di estrazione dalla pila S) — è esattamente ciò che trasforma la somma esplicita su tutte le coppie, di costo $\Theta(n^3)$, nella passata lineare $O(m)$ per sorgente dell'algoritmo di Brandes.

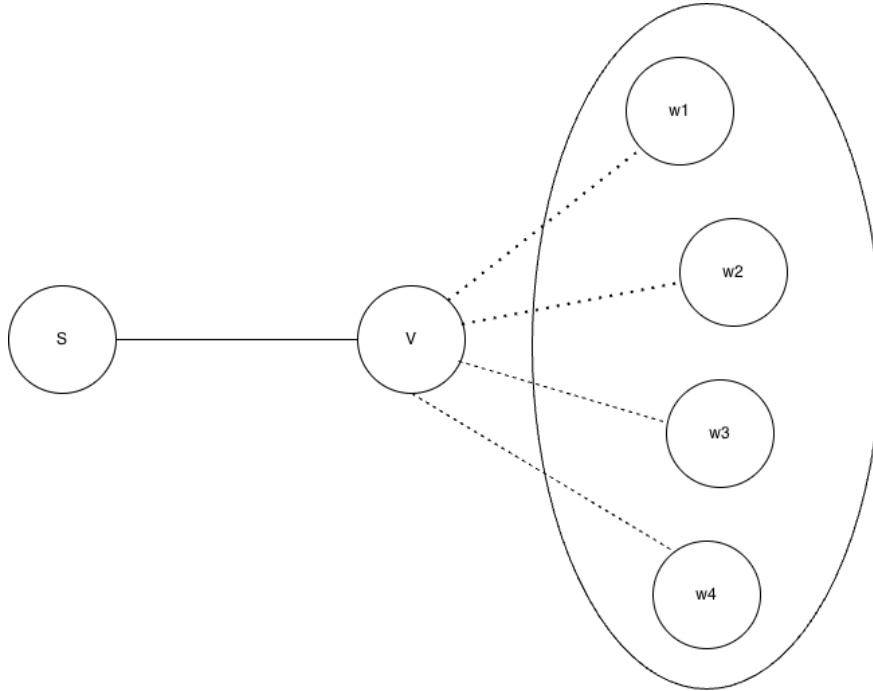


Figura 2.1: Struttura locale del DAG dei cammini minimi attorno al nodo v . La sorgente s raggiunge v tramite i cammini minimi già processati; da v si dipartono gli archi verso l'insieme dei successori $\{w_1, w_2, w_3, w_4\} = \{w : v \in P_s(w)\}$, cioè i nodi per cui v è predecessore diretto nel DAG.

La Figura 2.1 illustra la struttura locale sfruttata dalla ricorsione nel calcolo della dipendenza $\delta_s(v)$. Fissata una sorgente s , ogni cammino minimo che prosegue oltre v verso un target più lontano deve necessariamente transitare per uno dei successori di v nel DAG dei cammini minimi, qui indicati con $w_1, \dots, w_4$. Questi archi tratteggiati rappresentano quindi

una partizione dell'insieme dei target raggiungibili da v : ciascun target lontano contribuisce alla dipendenza $\delta_s(v)$ attraverso esattamente uno di questi successori. È questa proprietà a fondare la decomposizione ricorsiva (2.2); come osservato sopra, il contributo di ciascun w_i è già interamente noto al momento dell'elaborazione di v .

2.3 Pseudocodice

Algorithm 2 Betweenness centrality (Brandes, grafi non pesati)

```

1:  $C_B[v] \leftarrow 0$  per ogni  $v \in V$ 
2: for  $s \in V$  do
3:   inizializza pila  $S \leftarrow \emptyset$ , liste  $P[w] \leftarrow \emptyset$ 
4:    $\sigma[t] \leftarrow 0$ ,  $\sigma[s] \leftarrow 1$ 
5:    $d[t] \leftarrow -1$ ,  $d[s] \leftarrow 0$ 
6:   coda  $Q \leftarrow \{s\}$ 
7:   while  $Q \neq \emptyset$  do ▷ BFS
8:     estrai  $v$  da  $Q$ ; inserisci  $v$  in  $S$ 
9:     for ogni vicino  $w$  di  $v$  do
10:      if  $d[w] < 0$  then ▷  $w$  trovato per la prima volta
11:         $d[w] \leftarrow d[v] + 1$ ; inserisci  $w$  in  $Q$ 
12:      end if
13:      if  $d[w] = d[v] + 1$  then ▷ cammino minimo verso  $w$  via  $v$ 
14:         $\sigma[w] \leftarrow \sigma[w] + \sigma[v]$ 
15:        inserisci  $v$  in  $P[w]$ 
16:      end if
17:    end for
18:  end while
19:   $\delta[v] \leftarrow 0$  per ogni  $v$ 
20:  while  $S \neq \emptyset$  do ▷ accumulo all'indietro
21:    estrai  $w$  dalla cima di  $S$ 
22:    for ogni  $v \in P[w]$  do
23:       $\delta[v] \leftarrow \delta[v] + \frac{\sigma[v]}{\sigma[w]}(1 + \delta[w])$ 
24:    end for
25:    if  $w \neq s$  then  $C_B[w] \leftarrow C_B[w] + \delta[w]$ 
26:    end if
27:  end while
28: end for
29: per grafi non orientati:  $C_B[v] \leftarrow C_B[v]/2$  per ogni  $v$ 

```

2.4 Esempio passo-passo: il diamante a 5 vertici

Per rendere concreto lo pseudocodice si segue l'esecuzione dell'algoritmo su un piccolo grafo non pesato, mostrando l'evoluzione di tutti i vettori coinvolti. Si considera il grafo di Figura 2.2, con $n = 5$ vertici e gli archi

$$\{0, 1\}, \{0, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}.$$

La struttura a diamante fra 0, 1, 2, 3 è scelta deliberatamente: da 0 a 3 esistono *due* cammini minimi, 0–1–3 e 0–2–3, così da osservare il conteggio σ assumere un valore maggiore di 1.

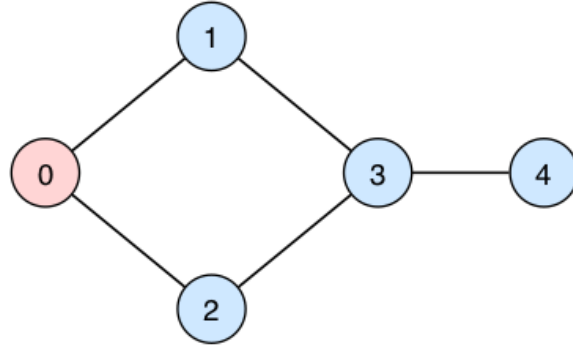


Figura 2.2: Il diamante a 5 vertici. In rosso la sorgente $s = 0$ da cui si avvia la visita.

Fase 1: il calcolo dei cammini minimi dalla sorgente $s = 0$

All'avvio $d[0] = 0$, $\sigma[0] = 1$, mentre per ogni altro vertice $d = -1$ (non visitato) e $\sigma = 0$; la coda contiene $\{0\}$. Le due tabelle seguenti riportano lo stato *successivo* all'estrazione di ciascun nodo dalla coda (il valore -1 indica un vertice non ancora raggiunto).

Tabella 2.1: Evoluzione di d (calcolo dei cammini minimi da $s = 0$).

nodo estratto	d_0	d_1	d_2	d_3	d_4
– init –	0	-1	-1	-1	-1
0	0	1	1	-1	-1
1	0	1	1	2	-1
2	0	1	1	2	-1
3	0	1	1	2	3
4	0	1	1	2	3

Tabella 2.2: Evoluzione di **sigma** (numero di cammini minimi).

nodo estratto	σ_0	σ_1	σ_2	σ_3	σ_4
– init –	1	0	0	0	0
0	1	1	1	0	0
1	1	1	1	1	0
2	1	1	1	2	0
3	1	1	1	2	2
4	1	1	1	2	2

Il passaggio cruciale è l'estrazione del nodo 2: poiché $d_3 = d_2 + 1$, il nodo 3 riceve un *secondo* contributo e σ_3 passa da 1 a 2, in corrispondenza dei due cammini minimi del diamante. Tale valore 2 si propaga poi al nodo 4. Al termine della visita la pila S contiene (dal fondo alla cima)

$$S = [0, 1, 2, 3, 4].$$

Evoluzione di Q e S

Le due strutture hanno ruoli complementari: la coda Q guida l'*ordine di scoperta* dei vertici, mentre la pila S ne *registra* l'ordine di visita per poterlo poi ripercorrere a ritroso nella fase di

accumulo. La Tabella 2.3 ne mostra l'evoluzione congiunta: a ogni passo si estrae un vertice v dalla testa di $\mathbf{Q}$, lo si inserisce in $\mathbf{S}$ e si aggiungono in coda a $\mathbf{Q}$ i soli vicini non ancora scoperti (quelli con $d = -1$). In tal modo ciascun vertice entra in $\mathbf{Q}$ una sola volta.

Tabella 2.3: Evoluzione di $\mathbf{Q}$ (coda FIFO) e $\mathbf{S}$ (pila) durante la visita da $s = 0$.

v estratto	vicini nuovi $\rightarrow \mathbf{Q}$	$\mathbf{Q}$ (testa $\rightarrow$ coda)	$\mathbf{S}$ (fondo $\rightarrow$ cima)
– init –	0	[0]	[]
0	1, 2	[1, 2]	[0]
1	3	[2, 3]	[0, 1]
2	–	[3]	[0, 1, 2]
3	4	[4]	[0, 1, 2, 3]
4	–	[]	[0, 1, 2, 3, 4]

Si nota come $\mathbf{Q}$ si riempia e si svuoti *contestualmente* (a ogni estrazione corrispondono zero o più inserimenti dei vicini nuovi), fino a svuotarsi del tutto a visita conclusa; $\mathbf{S}$, invece, si riempie soltanto, accumulando i vertici in ordine di distanza crescente. La fase di accumulo, descritta nel seguito, svuota poi $\mathbf{S}$ estraendo dalla cima: l'ordine risultante 4, 3, 2, 1, 0 è quello a distanza *decrescente*, esattamente come richiesto dalla relazione ricorsiva. È questa la ragione per cui si impiega una pila e non una seconda coda: la struttura LIFO inverte automaticamente l'ordine di visita.

Fase 2: accumulo delle dipendenze

Si svuota la pila $\mathbf{S}$ estraendo i nodi dalla cima (da 4 a 0). Per ogni nodo w si aggiornano i suoi predecessori v , quelli con $d_v = d_w - 1$, mediante

$$\delta_v += \frac{\sigma_v}{\sigma_w} (1 + \delta_w).$$

Tabella 2.4: Evoluzione di **delta** durante l'accumulo all'indietro ($s = 0$).

w corrente	δ_0	δ_1	δ_2	δ_3	δ_4
(start)	0	0	0	0	0
4	0	0	0	1	0
3	0	1	1	1	0
2	2	1	1	1	0
1	4	1	1	1	0

Dettaglio dei passaggi più significativi:

- $w = 4$: unico predecessore 3; $\delta_3 += \frac{\sigma_3}{\sigma_4}(1 + \delta_4) = \frac{2}{2}(1 + 0) = 1$.
- $w = 3$: due predecessori, 1 e 2; ciascuno riceve $\frac{\sigma_v}{\sigma_3}(1 + \delta_3) = \frac{1}{2}(1 + 1) = 1$. Qui la dipendenza si ripartisce fra i due cammini minimi alternativi.
- $w = 2$ e $w = 1$: il predecessore è la sorgente 0; ognuno aggiunge $\frac{1}{1}(1 + 1) = 2$, portando δ_0 a 4.

L'ultima estrazione, quella della sorgente 0, è omessa dalla tabella perché non produce alcun aggiornamento: la sorgente non ha predecessori (nessun vertice v con $d_v = d_0 - 1 = -1$)

e la sua dipendenza δ_0 non viene comunque sommata a C_B , in quanto $w = s$. Il contributo della sola sorgente $s = 0$ è quindi

$$C_B^{(s=0)} = [0, 1, 1, 1, 0].$$

Risultato finale

Ripetendo la procedura per tutte le sorgenti $s = 0, \dots, 4$, sommando i contributi e dividendo per 2 (grafo non orientato) si ottiene

$$C_B = [0.5, 1.0, 1.0, 3.5, 0.0].$$

Il nodo 3 presenta la betweenness nettamente maggiore: è il *ponte* del grafo, attraversato dalla quasi totalità dei cammini minimi. I nodi 1 e 2, simmetrici, hanno valore identico, mentre il nodo 4, foglia, ha betweenness nulla.

2.5 Generazione dei grafi di test

Le prove sperimentali richiedono la generazione di grafi casuali *connessi*: un grafo disconnesso falserebbe l'analisi dei tempi, in quanto il calcolo dei cammini minimi da una data sorgente esplorerebbe la sola componente cui essa appartiene. La funzione `genera_grafo` costruisce dapprima uno *spanning tree* casuale, che garantisce la connessione mediante $n - 1$ archi, e aggiunge successivamente archi casuali fino a raggiungere il numero m richiesto, escludendo cappi e archi multipli.

Listing 2.1: Generazione di un grafo connesso (estratto).

```

1 // 1) spanning tree casuale: ogni nodo i si collega a un nodo
  precedente
2 for (int i = 1; i < n; i++) {
3     std::uniform_int_distribution<int> prec(0, i - 1);
4     aggiungi(i, prec(gen));
5 }
6 // 2) archi restanti scelti a caso fino a m archi totali
7 std::uniform_int_distribution<int> dist(0, n - 1);
8 while ((int)g.U.size() < m) {
9     aggiungi(dist(gen), dist(gen));
10 }

```

La lambda `aggiungi` normalizza l'arco (u, v) con $u < v$, scarta i duplicati tramite un `std::set` e aggiorna le liste di adiacenza.

2.6 Il calcolo dei cammini minimi

Il blocco che realizza il calcolo dei cammini minimi determina, per ciascun vertice, la distanza `d` e il numero di cammini minimi `sigma`, e inserisce i vertici nell'ordine di visita nella pila `S` (la pila *S* dello pseudocodice). La coda dei nodi ancora da espandere è chiamata `Q`. Si osserva la piena corrispondenza con le righe 7–14 dello pseudocodice.

Per la pila *S* si è scelto `std::stack`, che corrisponde direttamente alla struttura dello pseudocodice. Una soluzione equivalente sarebbe un `std::vector` riempito durante la visita e percorso all'indietro: le due realizzazioni hanno la stessa complessità e, in pratica, prestazioni indistinguibili; si è preferito `std::stack` per la maggiore aderenza all'algoritmo descritto.

Listing 2.2: Calcolo dei cammini minimi con il relativo conteggio (BFS).

```

1 while (!Q.empty()) {
2     int v = Q.front();
3     Q.pop();
4     S.push(v);                                // inserimento nella pila
5     for (int w : g.adj[v]) {
6         if (d[w] == -1) {                      // primo ritrovamento
7             d[w] = d[v] + 1;
8             Q.push(w);
9         }
10        if (d[w] == d[v] + 1)                  // cammino minimo verso w
11            sigma[w] += sigma[v];
12    }
13 }

```

Il vettore **sigma** è dichiarato di tipo `long long`: su grafi di grandi dimensioni ed elevata densità il numero di cammini minimi può eccedere l'intervallo rappresentabile da `int`, e tale scelta previene l'overflow.

2.7 L'accumulo all'indietro

Terminato il calcolo dei cammini minimi, i vertici vengono esaminati in ordine di distanza non crescente — estraendoli dalla pila **S** — e si applica la relazione (2.2). Una scelta implementativa rilevante consiste nel *non* memorizzare esplicitamente le liste dei predecessori $P[w]$: poiché un vicino v di w ne è predecessore se e solo se $d[v] = d[w] - 1$, i predecessori sono ricalcolati direttamente a partire dal solo vettore **d**, evitando l'occupazione di memoria e le numerose allocazioni dinamiche che la gestione esplicita di $P[w]$ comporterebbe.

Listing 2.3: Accumulo delle dipendenze (predecessori ricalcolati).

```

1 while (!S.empty()) {
2     int w = S.top();
3     S.pop();
4     for (int v : g.adj[w]) {
5         if (d[v] == d[w] - 1)                  // v e' predecessore di w
6             delta[v] += (double)sigma[v] / sigma[w] * (1 + delta[w]);
7     }
8     if (w != s)
9         CB[w] += delta[w];
10 }

```

Al termine del ciclo su tutte le sorgenti, trattandosi di un grafo non orientato, ciascuna coppia risulta conteggiata due volte; il valore $CB[v]$ viene pertanto diviso per 2.

2.8 Ottimizzazioni implementative

L'implementazione adotta alcune ottimizzazioni che, pur lasciando invariata la complessità asintotica $O(nm)$, ne riducono la costante moltiplicativa:

- i buffer **d**, **sigma** e **delta** sono allocati *una sola volta* all'esterno del ciclo sulle sorgenti e, a ogni iterazione, vengono riscritti tramite `std::fill` anziché riallocati; analogamente la pila **S** è dichiarata una sola volta e riutilizzata, poiché la fase di accumulo la svuota completamente a fine sorgente. Ciò elimina n coppie di operazioni `malloc/free` per esecuzione;

- l’eliminazione delle liste di predecessori $P[w]$, illustrata in precedenza, rimuove l’allocazione dinamica più onerosa dell’algoritmo;
- per la frontiera della BFS si è mantenuto `std::queue` (basato su `std::deque`). È stata valutata l’alternativa di una coda FIFO ad array preallocato con due indici, che eviterebbe le allocazioni a blocchi del `deque`; il confronto sperimentale non ha però evidenziato alcun guadagno (con l’ottimizzazione -O2 il `deque` riusa i blocchi già allocati tra una sorgente e l’altra). Si è perciò preferito `std::queue` per la maggiore leggibilità;
- prima delle misurazioni viene eseguita una breve fase di *warm-up* — alcune esecuzioni preliminari su un grafo di dimensione intermedia, i cui risultati sono scartati — al fine di portare il processore alla frequenza di regime e di precaricare le memorie cache, attenuando l’artefatto di misura noto come “primo punto lento”.

Come mostrato nel Capitolo 4, l’output numerico rimane identico a quello della versione non ottimizzata, a meno del rumore in virgola mobile.

2.8.1 Rappresentazione dell’adiacenza: liste contro CSR

La rappresentazione adottata per le liste di adiacenza è un vettore di vettori (`std::vector<std::vector<int>>`) assimilabile a una matrice irregolare in cui ogni riga corrisponde a un vertice e gli elementi della riga sono i suoi vicini. Ciascuna riga è però un blocco di memoria allocato separatamente, e quindi potenzialmente lontano dagli altri: la scansione dei vicini durante la BFS comporta un salto verso un’area di memoria scollegata, con possibili *cache miss*.

Si è perciò valutata una rappresentazione alternativa, il formato *Compressed Sparse Row* (CSR), che sostituisce il vettore di vettori con due soli array contigui: un array `adj` che contiene tutti i vicini concatenati in un unico blocco, e un array `offset` che, per ciascun vertice v , individua l’intervallo di `adj` corrispondente ai suoi vicini (da `offset[v]` a `offset[v+1]-1`). L’intera adiacenza occupa così una sola zona di memoria, percorsa in modo sequenziale e prevedibile dal *prefetcher* hardware.

Sul piano teorico la CSR dovrebbe ridurre i *cache miss*, con vantaggio crescente sui grafi densi, dove la scansione dei vicini domina il costo. Il confronto sperimentale — a parità di algoritmo, cambiando unicamente la rappresentazione — non ha però evidenziato alcun guadagno misurabile: come mostrato in Figura 2.3, le due curve risultano sostanzialmente sovrapposte a ogni densità, con un rapporto medio dei tempi compreso tra 0,96 e 1,01. La ragione più plausibile è che l’ottimizzazione del compilatore (-O3) e il riuso dei blocchi di memoria già allocati tra una sorgente e l’altra rendano di fatto irrilevante la differenza di layout: il sistema di memoria sottostante riesce già a servire con efficienza anche l’accesso al vettore di vettori. Si tratta dunque di un’ottimizzazione *ipotizzata e sperimentalmente smentita*, analogamente a quanto osservato per la coda della BFS.

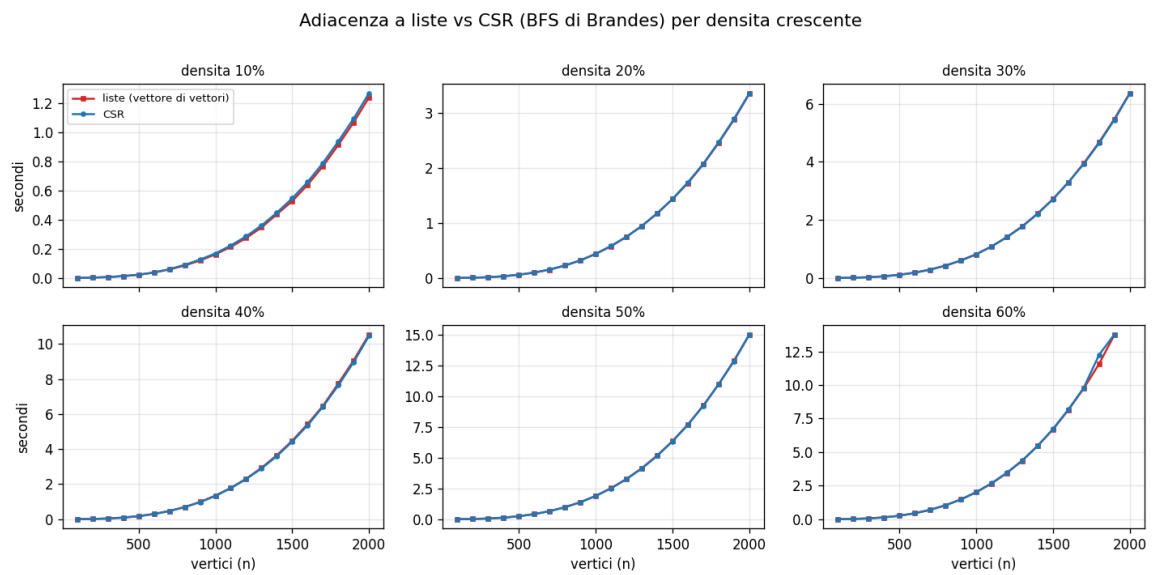


Figura 2.3: Tempo di esecuzione della BFS di Brandes con adiacenza a liste (vettore di vettori) e in formato CSR, per densità crescente. Le curve sono sostanzialmente sovrapposte: la CSR non porta un guadagno misurabile.

Capitolo 3

Estensione ai grafi pesati: l'algoritmo di Dijkstra

Quanto esposto finora si riferisce a grafi *non pesati*, in cui ogni arco ha costo unitario. In numerose applicazioni reali, tuttavia, agli archi è associato un peso, interpretabile come distanza, tempo o capacità. Il presente capitolo descrive come l'algoritmo di Brandes debba essere modificato per trattare i grafi pesati; la corrispondente implementazione è contenuta nel file `src/brandes_weighted.cpp`.

3.1 Inadeguatezza della BFS

La BFS determina correttamente i cammini minimi soltanto quando tutti gli archi hanno costo identico: in tal caso il primo istante in cui un vertice viene raggiunto corrisponde al suo cammino minimo, e il numero di archi coincide con la distanza. In presenza di pesi questa proprietà decade, poiché un cammino composto da più archi di peso ridotto può risultare complessivamente più breve di un singolo arco di peso elevato. La BFS, che procede per livelli di numero di archi, non individuerrebbe tali cammini.

La struttura generale dell'algoritmo di Brandes — elaborazione di una sorgente per volta, con una fase di calcolo dei cammini minimi seguita dall'accumulo all'indietro delle dipendenze — rimane invariata: a mutare è esclusivamente la prima fase, in cui la BFS è sostituita dall'algoritmo di **Dijkstra**. La relazione ricorsiva sulle dipendenze e la fase di accumulo conservano la medesima logica.

3.2 Implementazione con pesi non unitari

Rispetto alla versione non pesata si rendono necessarie tre modifiche.

1. **Calcolo delle distanze.** Il vettore `d` assume tipo `long long`, dovendo accumulare la somma dei pesi anziché un conteggio di livelli, ed è inizializzato a $+\infty$ in luogo di -1 ; il valore infinito è rappresentato dal massimo valore esprimibile dal tipo.

2. **Coda a priorità in luogo della coda FIFO.** La `std::queue` è sostituita da una `std::priority_queue` configurata come *min-heap*: in cima si trova costantemente il vertice di distanza provvisoria minima, ossia il successivo da finalizzare. Il modificatore `std::greater` inverte l'ordinamento predefinito — che fornirebbe il massimo — restituendo il minimo.

3. Azzeramento di sigma alla scoperta di cammini più brevi. Quando si individua un cammino *strettamente* più breve verso un vicino, i conteggi di cammini minimi accumulati fino a quel momento si riferiscono a una distanza ormai superata e perdono validità; il valore `sigma[w]` viene pertanto azzerato prima di riavviare il conteggio.

Listing 3.1: Fase 1 pesata: Dijkstra con conteggio dei cammini minimi.

```

1 while (!Q.empty()) {
2     auto [dv, v] = Q.top();           // dv: chiave estratta,
        ignorata
3     Q.pop();
4     if (nodi_completati[v]) continue; // voce obsoleta: scarta
5     nodi_completati[v] = 1;
6     S.push(v);
7     for (auto [w, peso] : g.adj[v]) {
8         long long nd = d[v] + peso;
9         if (nd < d[w]) {               // cammino piu' corto
10            d[w] = nd;
11            sigma[w] = 0;               // i conteggi vecchi non sono
                piu' minimi
12            Q.push({nd, w});
13        }
14        if (nd == d[w])                 // altro cammino minimo verso
            w
15            sigma[w] += sigma[v];
16    }
17 }

```

3.3 La cancellazione pigra

La `std::priority_queue` della libreria standard non mette a disposizione l'operazione di *decrease-key*, ossia l'aggiornamento della priorità di un elemento già inserito. Si ricorre pertanto alla tecnica della *cancellazione pigra* (*lazy deletion*): all'individuazione di una distanza migliore per un vertice si inserisce una nuova coppia (d , vertice), lasciando inalterata la voce precedente. Al momento dell'estrazione, qualora il vertice risulti già finalizzato (in base al flag `nodi_completati`), la voce è obsoleta e viene scartata mediante `continue`. In tal modo ciascun vertice è inserito nella pila `S` una sola volta, secondo l'ordine di distanza crescente, indispensabile per la successiva fase di accumulo.

La correttezza di questo schema — finalizzare un vertice alla sua *prima* estrazione e scartare le copie successive — è dovuta appunto dal min-heap: nel momento in cui un vertice v viene estratto per la prima volta, la sua distanza provvisoria è *già* quella minima definitiva, e nessun cammino più breve potrà essere scoperto in seguito. Si osservi che questa proprietà dipende in modo essenziale dall'ipotesi di **pesi non negativi**: con archi di peso negativo un cammino più breve potrebbe emergere *dopo* la finalizzazione di un vertice, e l'algoritmo di Dijkstra — cancellazione pigra inclusa — fornirebbe risultati errati. È precisamente per questo motivo che Dijkstra richiede pesi non negativi; in presenza di pesi negativi occorre ricorrere ad algoritmi diversi, come quello di Bellman–Ford.

3.4 L'accumulo all'indietro nel caso pesato

La fase di accumulo è strutturalmente identica a quella del caso non pesato: i vertici sono estratti dalla pila `S` e la dipendenza è distribuita ai predecessori. L'unica differenza risiede

nella *condizione di predecessore*: nel caso non pesato un vicino v di w ne è predecessore se $d_v = d_w - 1$, mentre nel caso pesato la condizione diviene

$$d_v + \text{peso}(v, w) = d_w. \quad (3.1)$$

Analogamente al caso non pesato, i predecessori sono ricalcolati a partire dalle distanze, senza il mantenimento di liste esplicite.

Listing 3.2: Accumulo pesato: cambia solo la condizione di predecessore.

```

1 while (!S.empty()) {
2     int w = S.top();
3     S.pop();
4     for (auto [v, peso] : g.adj[w])
5         if (d[v] + peso == d[w])           // v e' predecessore di w
6             delta[v] += (double)sigma[v] / sigma[w] * (1 + delta[w]);
7     if (w != s) CB[w] += delta[w];
8 }

```

3.5 Complessità

Con la coda a priorità l'estrazione del minimo e l'inserimento hanno costo $O(\log n)$ ciascuno e si effettuano $O(m)$ aggiornamenti per sorgente; la singola esecuzione di Dijkstra costa pertanto $O(m \log n)$, a fronte dell' $O(m)$ della BFS. Esteso a tutte le n sorgenti, il costo complessivo risulta

$$O(nm \log n) \quad (3.2)$$

nel caso di heap binario. L'impiego di un *Fibonacci heap* [4], che ammortizza l'operazione di *decrease-key* a $O(1)$, consente di raggiungere il costo teorico $O(nm + n^2 \log n)$ indicato da Brandes. Il fattore logaritmico aggiuntivo rappresenta il costo della maggiore generalità: il caso pesato è intrinsecamente più oneroso di quello non pesato. La Figura 3.1 riporta i tempi misurati per la versione pesata, secondo lo stesso schema sperimentale adottato nel caso non pesato — una curva per ciascuna densità, con sweep su n —; l'andamento conferma il maggiore costo imputabile alla coda a priorità.

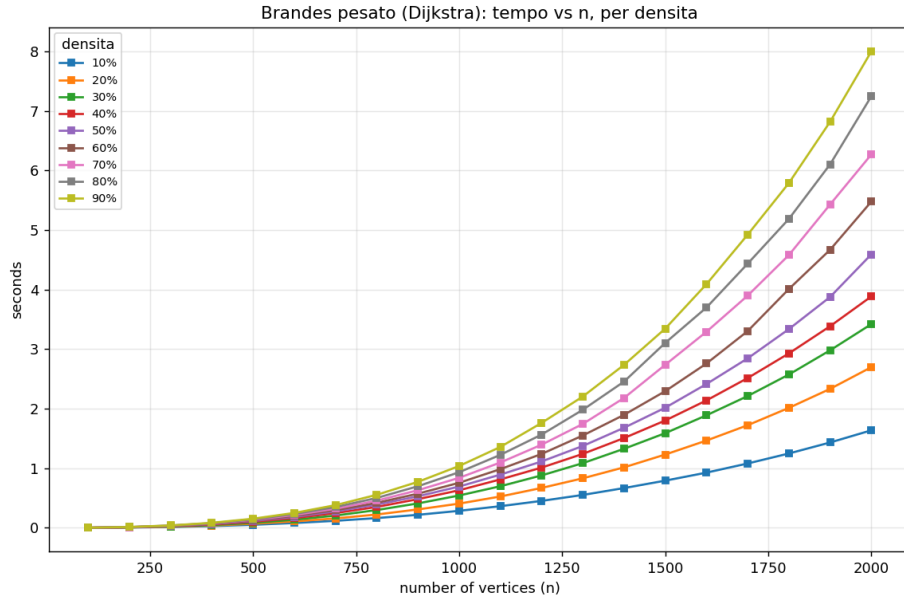


Figura 3.1: Tempo di esecuzione della versione pesata (Dijkstra) in funzione del numero di vertici, una curva per ciascuna densità.

3.6 Verifica ed esempio visivo

Anche la versione pesata è stata validata mediante confronto dei valori di C_B con quelli prodotti da NetworkX, invocata con l'opzione `weight='weight'` oltre a `normalized=False`: i risultati coincidono. La Figura 3.2 riporta un grafo pesato di esempio con i nodi-ponte evidenziati; le etichette sugli archi ne indicano i pesi.

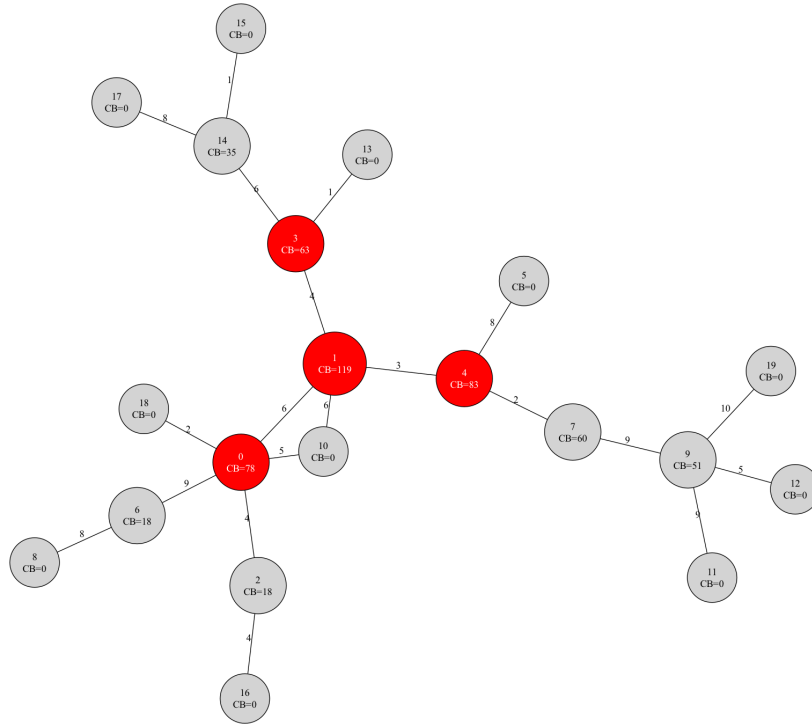


Figura 3.2: Grafo pesato di esempio: in rosso i nodi con betweenness più alta. Le etichette sugli archi indicano i pesi. Rispetto al caso non pesato, i pesi modificano la struttura dei cammini minimi e, di conseguenza, l'insieme dei nodi che vi giacciono.

Capitolo 4

Risultati sperimentali

4.1 Impostazione degli esperimenti

Le prove sono state condotte generando grafi casuali connessi mediante la funzione `genera_grafo` descritta nel Capitolo 2. Al fine di contenere il rumore di misura sono stati adottati i seguenti accorgimenti:

- ciascuna configurazione (n, m) è stata cronometrata ripetendo la misura cinque volte sullo stesso grafo e assumendo il tempo *minimo*: la corsa più veloce è quella meno disturbata dalle interferenze del sistema operativo (scheduling, altri processi), e fornisce dunque la stima più pulita del costo effettivo dell'algoritmo;
- i tempi sono stati rilevati tramite `std::chrono::high_resolution_clock`, cronometrando la sola invocazione di `brandes`;
- è stata anteposta una fase di *warm-up* per stabilizzare la frequenza del processore.

I risultati sono salvati in un file `risultati.csv` ed elaborati da uno script Python (`plot.py`) per la produzione dei grafici; la variante pesata segue il medesimo protocollo, con output su `risultati_w.csv`.

4.2 La pipeline sperimentale

L'intera raccolta dei dati è automatizzata: nessun grafo viene letto da file né prodotto a mano. Ogni eseguibile genera internamente i grafi, cronometra l'algoritmo e scrive un file `.csv`, che uno script Python trasforma poi in figura. Il flusso, identico per le tre varianti, si articola in cinque stadi.

1. **Sweep dei parametri.** Due cicli annidati esplorano le configurazioni: la densità d dal 10% al 90% a passi del 10% e il numero di vertici n da 100 a 2000 a passi di 100. Per ciascuna coppia il numero di archi è fissato a $m = \lfloor d \cdot \binom{n}{2} \rfloor$; le configurazioni con $m < n - 1$ (troppo pochi archi per essere connesse) vengono saltate.
2. **Generazione del grafo.** Per ogni (n, m) la funzione `genera_grafo(n, m, seed)` costruisce un grafo casuale connesso (spanning tree + archi aggiuntivi, Capitolo 2). Il seme è fisso, così che l'esperimento sia *riproducibile*: rilanciando i programmi si ottengono gli stessi grafi e quindi gli stessi tempi a meno del rumore di misura.
3. **Warm-up.** Prima del ciclo di misura l'algoritmo gira alcune volte su un grafo intermedio, scartandone i risultati, per portare il processore alla frequenza di regime e precaricare le cache.

4. **Cronometraggio.** La sola chiamata a `brandes` è misurata con `std::chrono`, ripetuta cinque volte; si conserva il tempo *minimo*, la corsa meno disturbata dal sistema operativo.
5. **Scrittura del CSV.** Per ogni configurazione viene emessa una riga `n,m,densita,tempo_us` (tempo in microsecondi). Il file risultante è l'unico ingresso degli script Python, che raggruppano le righe per densità e tracciano il tempo in funzione di n (o di $n \cdot m$).

Oltre allo sweep, gli eseguibili non pesato e pesato generano un piccolo grafo di esempio ($n = 20$), ne evidenziano i nodi-ponte (primo 20% per betweenness) e ne scrivono la descrizione Graphviz (`.dot`), resa poi in immagine da `neato` — è l'origine delle Figure 4.1 e 3.2.

La corrispondenza fra programma, file prodotto e figura della tesina è riassunta nella Tabella 4.1.

Programma	Algoritmo	File <code>.csv</code>	Figure prodotte
<code>brandes</code>	Brandes (BFS)	<code>risultati.csv</code>	4.2, 4.3
<code>brandes_weighted</code>	Brandes (Dijkstra)	<code>risultati_w.csv</code>	3.1
<code>standard</code>	classico $O(n^3)$	<code>risultati_standard.csv</code>	4.3

Tabella 4.1: Pipeline: ogni eseguibile produce un CSV; gli script Python (`plot.py`, `confronto_densita.py`) li combinano nelle figure.

4.3 Verifica di correttezza

Prima di procedere all'analisi dei tempi è indispensabile accertare la correttezza dei valori calcolati dall'implementazione. I valori di C_B prodotti dal codice C++ sono stati confrontati con quelli forniti dalla funzione `betweenness_centrality` della libreria NetworkX [5] (con `normalized=False`, così da adottare la medesima scala) su un grafo di prova con $n = 15$ e $m = 20$.

Tabella 4.2: Confronto C_B : implementazione C++ vs NetworkX (estratto).

nodo	C++	NetworkX	differenza
1	36.3333	36.3333	3.3×10^{-11}
3	36.3333	36.3333	3.3×10^{-11}
0	33.6667	33.6667	3.3×10^{-11}
5	16.3333	16.3333	3.3×10^{-11}
11	0.0000	0.0000	0.0
13	0.3333	0.3333	3.3×10^{-13}

La differenza massima riscontrata sull'insieme dei nodi è dell'ordine di 10^{-11} , riconducibile al solo rumore in virgola mobile: le due implementazioni producono il medesimo risultato. La verifica è stata ripetuta sulla versione ottimizzata del codice (buffer riutilizzati, predecessori ricalcolati), con esito parimenti coincidente, a conferma del fatto che le ottimizzazioni implementative non ne alterano la correttezza.

4.4 Interpretazione visiva della betweenness

Allo scopo di attribuire un significato intuitivo alla misura, la Figura 4.1 mostra un grafo di esempio in cui i nodi con betweenness più elevata — i cosiddetti *ponti* — sono evidenziati in rosso.

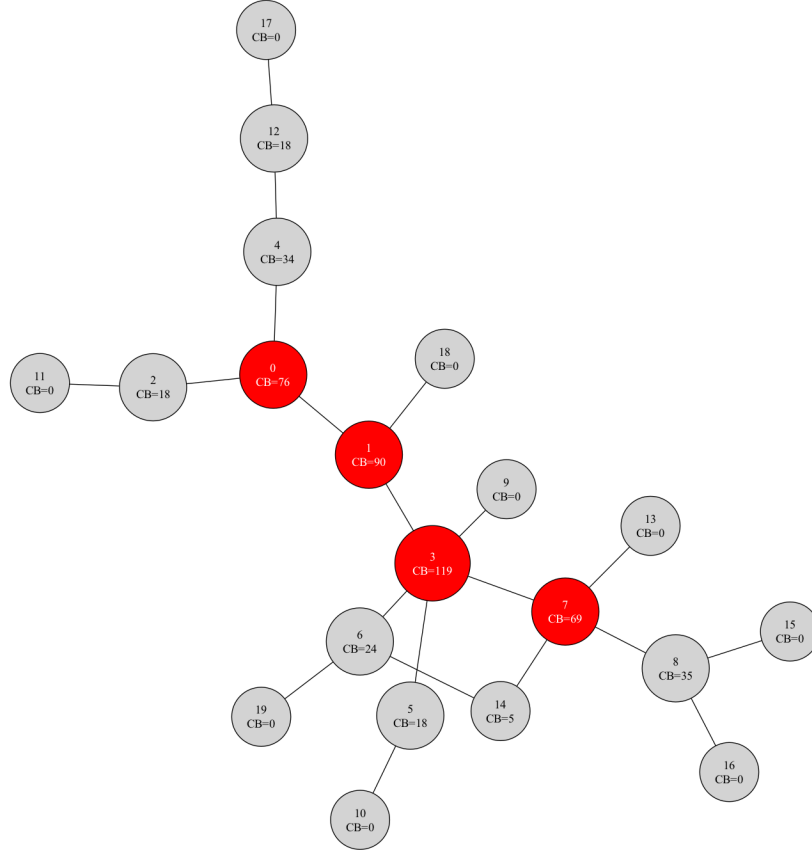


Figura 4.1: Grafo di esempio: in rosso i nodi con betweenness più alta. Si nota come i ponti non siano necessariamente i nodi con più connessioni, ma quelli che giacciono sui passaggi obbligati tra le diverse zone del grafo. I nodi foglia, come il nodo 11, hanno betweenness nulla.

4.5 Analisi della complessità

Per verificare la complessità $O(nm)$ è stato misurato il tempo di esecuzione al variare di n e m . Riportando il tempo in funzione del prodotto $n \cdot m$ si ottiene un andamento sostanzialmente lineare, con coefficiente di determinazione $R^2 \approx 0.93$. Una rappresentazione complementare, con il tempo in funzione di n per densità fissata, è riportata nella Figura 4.2 della sezione successiva.

4.5.1 Il sovraccarico lineare per sorgente

Un'osservazione rilevante emersa dagli esperimenti riguarda il comportamento a m fissato: mantenendo costante il numero di archi e incrementando n , il tempo non cresce in modo lineare bensì quadratico. La causa risiede nel fatto che il costo effettivo per ciascuna sorgente non è $O(m)$ ma $O(n + m)$, poiché l'inizializzazione dei buffer (`d`, `sigma`, `delta`) richiede $O(n)$.

operazioni a ogni sorgente. Il costo complessivo è dunque

$$O(n \cdot (n + m)) = O(n^2 + nm), \quad (4.1)$$

e quando m è piccolo rispetto a n il termine n^2 risulta dominante. Ciò giustifica la curvatura osservata e mette in evidenza come la stima $O(nm)$ presupponga implicitamente grafi sufficientemente densi ($m \gtrsim n$).

4.6 Riproduzione dell'esperimento di Brandes

Infine, è stato riprodotto lo schema sperimentale dell'articolo originale: uno sweep del numero di vertici da 100 a 2000, con una curva per ciascuna densità compresa tra il 10% e il 90%, secondo lo stesso impianto della Figura 3 di Brandes [1]. Il risultato è riportato nella Figura 4.2.

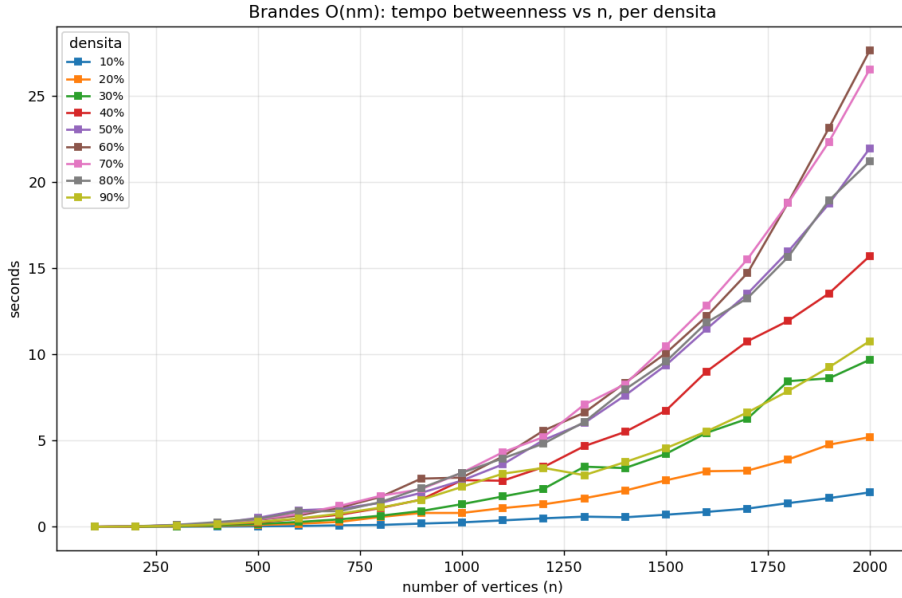


Figura 4.2: Tempo di esecuzione (caso non pesato) in funzione del numero di vertici, una curva per ciascuna densità dal 10% al 90%: riproduzione della Figura 3 dell'articolo originale di Brandes.

La rappresentazione per densità non è una semplice scelta estetica: ciascuna curva fissa la densità, ossia un valore di m proporzionale a n^2 , e isola così l'effetto del numero di archi sul tempo. A parità di n , passando da una curva all'altra cambia esclusivamente m ; lo scostamento verticale fra le curve misura perciò in modo diretto la dipendenza da m insita nel fattore $O(nm)$. Si osserva infatti che, fissato n , il tempo cresce con la densità per gran parte dell'intervallo, come atteso da una complessità lineare in m .

Alle densità più elevate, però, l'andamento si inverte e il tempo torna a diminuire pur continuando m a crescere. Il fenomeno è sistematico — si ripresenta identico a $n = 1000$ e $n = 2000$ — e ha una spiegazione strutturale: nei grafi quasi completi il diametro si riduce a 1–2 e quasi tutti i vertici giacciono a distanza unitaria dalla sorgente. Di conseguenza pochissimi archi soddisfano la condizione di cammino minimo $d_w = d_v + 1$, e crollano sia gli aggiornamenti del conteggio σ sia, soprattutto, gli accumuli di dipendenza δ , che costituiscono le operazioni aritmeticamente più onerose. Il numero di archi *esaminati* resta $\Theta(nm)$, ma il lavoro *effettivo* per arco diventa quasi nullo, e il tempo cala. Le curve, dunque, non si limitano

a confermare l'andamento teorico: rendono evidente che il costo dipende dalla *struttura* dei cammini minimi e non dal solo numero di archi, motivo per cui la densità è il parametro più significativo da fare variare.

4.7 Confronto sperimentale con l'approccio standard

L'analisi precedente riguarda il solo algoritmo di Brandes; per quantificarne il vantaggio è stato affiancato, sui medesimi grafi, l'approccio standard $O(n^3)$ descritto nel Capitolo 1. La Figura 4.3 riporta, per ciascuna densità, le due curve sovrapposte in funzione di n sul range comune (fino a $n = 1000$, oltre il quale l' $O(n^3)$ diventa proibitivo sia per il tempo sia per la memoria $O(n^2)$). La scelta di confrontare *a parità di densità* è proprio ciò che rende leggibile il risultato: è la densità a determinare il regime in cui ci si trova.

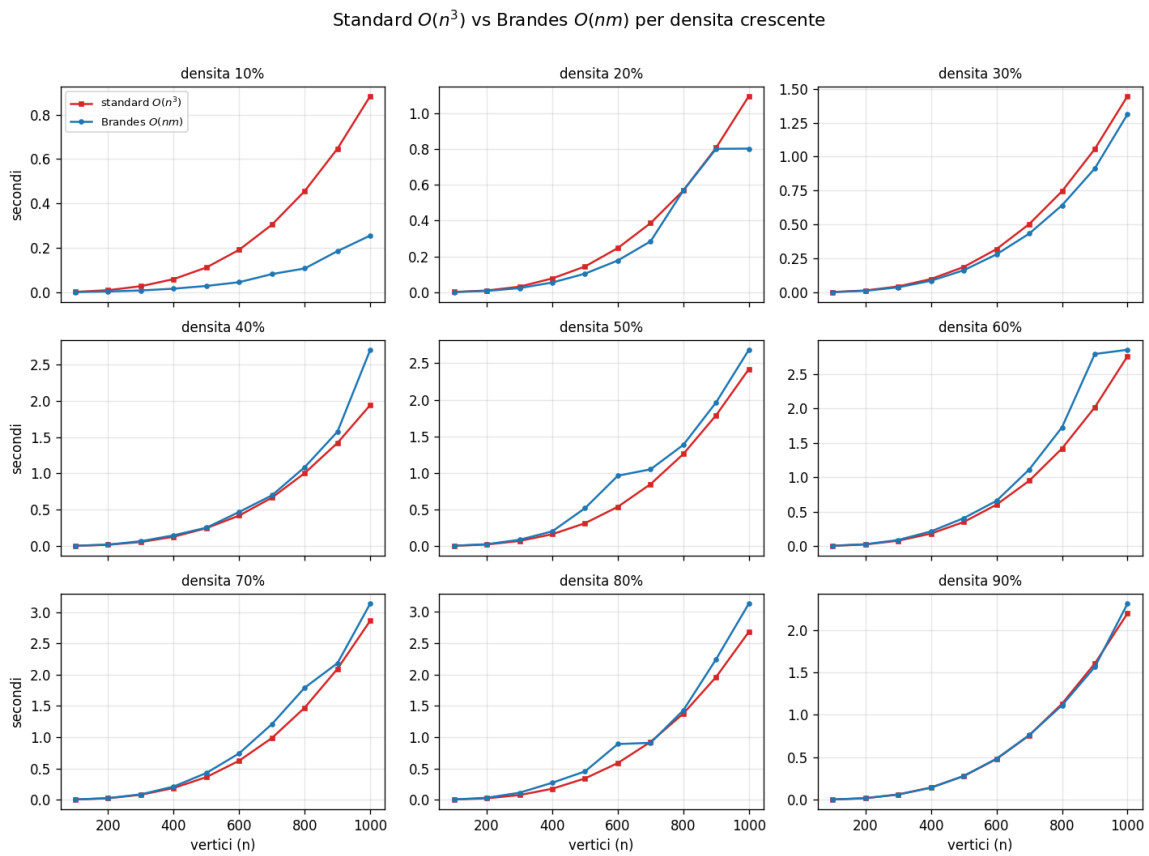


Figura 4.3: Confronto fra l'approccio standard $O(n^3)$ (rosso) e l'algoritmo di Brandes $O(nm)$ (blu) in funzione di n , un pannello per ciascuna densità dal 10% al 90%. Alle basse densità Brandes è nettamente più rapido; al crescere della densità le due curve diventano dello stesso ordine di grandezza.

Il confronto chiarisce il ruolo della densità. L'approccio standard è dominato dal triplo ciclo sulle coppie, di costo $\Theta(n^3)$ *a prescindere* dal numero di archi; il suo tempo varia perciò poco con la densità. La lieve diminuzione che si osserva alle densità più alte è imputabile alla seconda fase: nei grafi quasi completi i cammini minimi sono brevissimi e il criterio di Bellman è soddisfatto da pochi vertici intermedi, riducendo le operazioni di accumulo.

L'algoritmo di Brandes, di costo $O(nm)$, dipende invece direttamente da m . Nei grafi *sparsi* (densità 10–30%) il numero di archi è ridotto ed esso risulta nettamente più veloce:

a $n = 1000$ il vantaggio raggiunge un fattore prossimo a 3. Al crescere della densità m tende a $\Theta(n^2)$ e il costo $O(nm)$ degenera a $O(n^3)$: nelle densità intermedie e alte le due curve si intrecciano e restano dello stesso ordine di grandezza. Alle densità più estreme, anzi, entrambe tornano ad accelerare per il medesimo motivo strutturale già richiamato — la quasi-completezza del grafo rende i cammini minimi triviali e riduce drasticamente le operazioni di accumulo in entrambi gli algoritmi — come discusso a proposito della Figura 4.2. Il vantaggio robusto di Brandes va dunque cercato nei grafi sparsi — quelli tipici delle reti reali — e, soprattutto, nello *spazio*: $O(n + m)$ lineare contro l' $O(n^2)$ dello standard, a qualunque densità.

Capitolo 5

Conclusioni

5.1 Sintesi del lavoro

In questa tesina è stato implementato in C++ l'algoritmo di Brandes per il calcolo della betweenness centrality su grafi non orientati, nelle due varianti *non pesata* (BFS) e *pesata* (Dijkstra). Il lavoro ha riguardato quattro aspetti complementari:

- **correttezza:** i valori prodotti coincidono con quelli della libreria di riferimento NetworkX a meno del rumore in virgola mobile ($\sim 10^{-11}$), sia nella versione base sia in quella ottimizzata, e anche nella variante pesata (confronto con `weight='weight'`);
- **efficienza:** l'analisi sperimentale conferma la complessità $O(nm)$ prevista dalla teoria per il caso non pesato, evidenziando inoltre il sovraccarico lineare per sorgente che conduce al costo complessivo $O(n^2 + nm)$;
- **estensione ai grafi pesati:** la sostituzione della BFS con l'algoritmo di Dijkstra e una coda a priorità consente di trattare archi con peso arbitrario, a fronte di un fattore logaritmico aggiuntivo ($O(nm \log n)$ con heap binario), come confermato dai tempi misurati al variare della densità;
- **ottimizzazione:** il riutilizzo dei buffer di memoria e l'eliminazione delle liste di predecessori riducono la costante moltiplicativa senza modificare la complessità asintotica né i risultati.

In conclusione, l'algoritmo di Brandes si conferma una soluzione elegante ed efficiente al problema della betweenness centrality: il passaggio da uno spazio $O(n^2)$ a uno spazio lineare $O(n+m)$, ottenuto grazie all'accumulo ricorsivo delle dipendenze, è ciò che ne rende possibile l'uso su reti di grande dimensione.

Bibliografia

- [1] U. Brandes, *A Faster Algorithm for Betweenness Centrality*, Journal of Mathematical Sociology, 25(2):163–177, 2001.
- [2] L. C. Freeman, *A Set of Measures of Centrality Based on Betweenness*, Sociometry, 40(1):35–41, 1977.
- [3] J. M. Anthonisse, *The Rush in a Directed Graph*, Stichting Mathematisch Centrum, Amsterdam, 1971.
- [4] M. L. Fredman, R. E. Tarjan, *Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms*, Journal of the ACM, 34(3):596–615, 1987.
- [5] A. Hagberg, D. Schult, P. Swart, *Exploring Network Structure, Dynamics, and Function using NetworkX*, Proceedings of the 7th Python in Science Conference, 2008.